

ZeroTrace: A Cryptographically Verifiable and Error-Resilient Text Watermarking Engine with Graded Tamper Telemetry

Sreephaneesha Kanugovi

Computer Science Engineering (AIML)
PES University, Bengaluru, India
SRN: PES1UG23AM314
Email: pes1ug23am314@pes.edu

Vivek Gowda S

Computer Science Engineering (AIML)
PES University, Bengaluru, India
SRN: PES1UG23AM355
Email: pes1ug23am355@pes.edu

Chandan R

Computer Science Engineering (AIML)
PES University, Bengaluru, India
SRN: PES1UG23AM917
Email: pes1ug23am917@pes.edu

Dr. Vidhya K (Mentor)

Computer Science Engineering
PES University, Bengaluru, India
Email: vidhyak@pes.edu

Abstract—The proliferation of Large Language Models (LLMs) has accelerated content creation but simultaneously introduced critical challenges in verifying content authenticity, ensuring non-repudiation, and identifying downstream text tampering. Traditional statistical watermarking techniques alter token generation probabilities but degrade visual quality, require vocabulary-level model access, and fail under semantic paraphrasing. Conversely, basic steganographic approaches are fragile, breaking under minor edits due to frame-shift synchronization loss. This paper proposes ZeroTrace, a cryptographically secure, visually imperceptible, and error-resilient text watermarking framework. ZeroTrace integrates asymmetric digital signatures (RSA-2048) and document binding (SHA-256) with a concatenated channel coding scheme consisting of an outer Reed-Solomon code over $GF(2^8)$ and an inner BCH code. Embedding locations at word boundaries are randomized using an HMAC-SHA256 placement engine keyed by a server secret, document hash, and nonce, hiding the invisible zero-width Unicode bits from adversarial detection. Upon extraction, a character-level Longest Common Subsequence (LCS) alignment corrects frame-shifts caused by deletions or insertions. The system reconstructs the damaged watermark, verifies its cryptographic authenticity, and yields a graded tamper telemetry score mapping localized editing severity. Experimental evaluations demonstrate that ZeroTrace achieves 100% recovery under standard copy-paste operations and successfully corrects burst text deletions and typographical modifications.

Index Terms—AI Watermarking, Steganography, Asymmetric Cryptography, Reed-Solomon Codes, BCH Codes, Longest Common Subsequence, Provenance.

I. INTRODUCTION

As Large Language Models (LLMs) achieve human-like fluency, AI-generated text is widely deployed to author publications, documentation, articles, and reports. However, the ease of automated text generation has raised major challenges concerning misinformation, plagiarism, and intellectual property theft. Consequently, establishing text origin (provenance) and identifying alterations (tampering) is essential for trustworthy information distribution.

Existing text watermarking strategies fall into three paradigms:

- 1) **Statistical Token Watermarking**: Alters the probability distribution of generated tokens during decoding by splitting the vocabulary into “green” and “red” lists. While effective, it requires direct control over the LLM decoding engine, degrades writing fluency, and fails to survive moderate paraphrasing.
- 2) **Syntactic and Lexical Watermarking**: Substitutes synonyms or alters grammatical structures. These methods frequently modify the semantic intent of the text, degrade readability, and are easily identified by style classifiers.
- 3) **Steganographic Watermarking**: Encodes metadata directly inside the text using invisible Unicode characters. These approaches are visually imperceptible but fragile; deleting even a single character can shift bit coordinates, destroying the watermark payload.

To overcome these limitations, we introduce **ZeroTrace**, a visual-transparency-preserving, cryptographically secure watermarking engine that provides graded tamper diagnostics. ZeroTrace achieves security, imperceptibility, and robustness by combining asymmetric cryptography with concatenated channel codes. By binding the document’s SHA-256 hash directly to the signature, the system resists watermark transfer attacks. Furthermore, a character-level Longest Common Subsequence (LCS) alignment maps character insertions and deletions to symbol erasures, allowing the error-correcting codes to recover the provenance signature.

The primary contributions of this work are:

- **Asymmetric Non-Repudiation**: Integrating RSA-2048 signatures with SHA-256 document hashing, ensuring that watermarked text cannot be forged or transferred to other documents.

- **Concatenated Algebraic Codes:** Designing a nested ECC pipeline using a byte-oriented outer Reed–Solomon code to correct burst deletions and a bit-oriented inner BCH code to correct scattered typographical edits.
- **Keyed Placement Secrecy:** Implementing HMAC-SHA256 pseudorandom word-boundary mapping, preventing adversarial detection or targeted scrubbing of the invisible bits.
- **Alignment-driven Robustness:** Applying character-level LCS dynamic programming to solve frame-shift errors, transforming deletions and modifications into correctable erasures.
- **Tamper Telemetry:** Exposing a quantitative Integrity Score derived from the active ECC corrections required during decoding.

II. SYSTEM ARCHITECTURE

The ZeroTrace architecture consists of six modular pipeline stages divided into the Embedding Pipeline and the Extraction Pipeline. The visual representations of these workflows are detailed in Fig. 1 and Fig. 2.

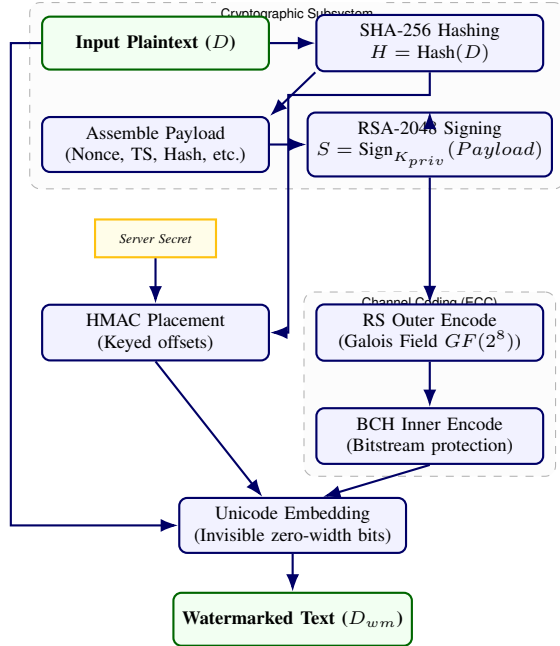


Fig. 1. ZeroTrace Watermark Embedding Pipeline Architecture.

The core modules operate as follows:

- **Canonicalization:** Standardizes whitespaces, tabs, and line endings in the document to ensure consistent hash computations and avoid false positives during verification.
- **Cryptographic Core:** Generates and handles RSA-2048 key pairs. Serializes the metadata payload, hashes the canonicalized text with SHA-256, and signs the resulting package using the private key.
- **Bitstream Serializer:** Converts the signed payload (stored in a JSON wrapper containing payload string and signature) into a raw bitstream.

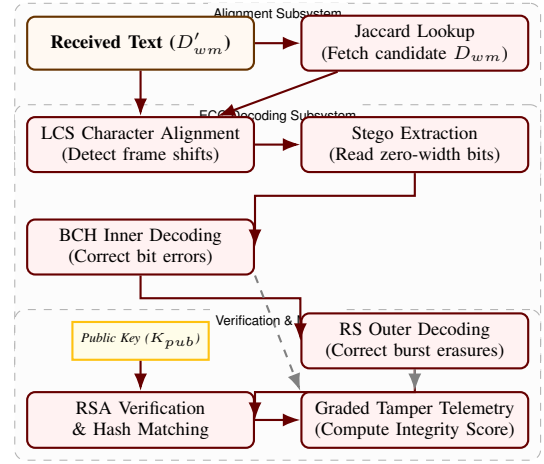


Fig. 2. ZeroTrace Watermark Extraction and Verification Pipeline Architecture.

- **Concatenated ECC Codec:** Takes the bitstream, runs it through the Reed–Solomon encoder, and subsequently processes it using the BCH encoder. The inverse transformations are performed during extraction.
- **Keyed Placement Engine:** Uses an HMAC-SHA256 generator initialized with the server secret key. The engine hashes the combination of the document hash and nonce to compute pseudorandom offset indexes corresponding to word boundaries in the text.
- **Unicode Steganographic Channel:** Interacts directly with the text strings, inserting or reading zero-width characters (`\u200B` for 0 and `\u200C` for 1) at the computed boundaries.

III. MATHEMATICAL FORMULATIONS

This section details the mathematical models governing ZeroTrace’s cryptographic security, Galois Field arithmetic, error-correcting codes, and structural alignment.

A. Document Binding and Asymmetric Signatures

Let D denote the clean canonicalized plaintext document. The document binding hash $H \in \{0, 1\}^{256}$ is computed as:

$$H = \text{SHA256}(D) \quad (1)$$

The provenance payload P is structured as a tuple:

$$P = (\text{version}, \text{provider}, \text{modelId}, \text{timestamp}, \text{nonce}, H) \quad (2)$$

The serialized string representation of P , denoted as $\bar{P}$, is signed using the RSA-2048 private exponent d and modulus n :

$$S = \text{Sign}_{K_{priv}}(\bar{P}) \equiv \text{SHA256}(\bar{P})^d \pmod{n} \quad (3)$$

The digital signature verification step using the public exponent e validates:

$$\text{SHA256}(\bar{P}) \equiv S^e \pmod{n} \quad (4)$$

If Eq. (4) holds, and the extracted document hash H matches the SHA-256 hash of the received stripped document D' , the

watermark is cryptographically authentic and bound to the document.

B. Galois Field $GF(2^8)$ Arithmetic

To compute Reed–Solomon parity bytes, operations must be performed over the Galois Field $GF(2^8)$. Field elements are represented as polynomials of degree less than 8 over $GF(2)$. Arithmetic is performed modulo the primitive generator polynomial:

$$p(x) = x^8 + x^4 + x^3 + x^2 + 1 \quad (5)$$

- **Addition/Subtraction:** Equivalent to bitwise exclusive-OR (XOR) operations:

$$a(x) \oplus b(x) = \sum_{i=0}^7 (a_i \oplus b_i) x^i \quad (6)$$

- **Multiplication:** Handled via exponentiation tables generated using a generator element $\alpha = 2$ (00000010₂). For any non-zero elements $A = \alpha^i$ and $B = \alpha^j$:

$$A \otimes B = \alpha^{(i+j) \pmod{255}} \quad (7)$$

C. Reed–Solomon Outer Code

The outer code is an (N, K) Reed–Solomon code over $GF(2^8)$. The message of length K is treated as a polynomial $m(x) = \sum_{i=0}^{K-1} m_i x^i$. The systematic encoder computes parity symbols as:

$$p(x) = x^{N-K} m(x) \pmod{g(x)} \quad (8)$$

where $g(x)$ is the generator polynomial defined by:

$$g(x) = \prod_{i=0}^{N-K-1} (x - \alpha^i) \quad (9)$$

The resulting code block is $c(x) = x^{N-K} m(x) + p(x)$. For $2t = N - K = 16$ parity symbols, the Reed–Solomon outer code can correct up to $t = 8$ byte symbol errors per block.

During decoding, syndromes S_i are computed from the received block $r(x)$:

$$S_i = r(\alpha^i), \quad i = 0, 1, \dots, 2t - 1 \quad (10)$$

The Berlekamp–Massey algorithm is used to find the error locator polynomial $\Lambda(x)$, whose roots indicate the error locations. The error values are calculated using Forney’s formula:

$$Y_j = -\frac{\Omega(\beta_j^{-1})}{\Lambda'(\beta_j^{-1})} \quad (11)$$

where $\Omega(x)$ is the error-evaluator polynomial and β_j^{-1} represents the error locations.

D. BCH Inner Code

The inner code is a binary cyclic $BCH(n, k, t)$ code, where $n = 2^m - 1$. For $m = 8$, we configure a $BCH(255, 191, 8)$ code. The generator polynomial $g_{bch}(x)$ is computed as the least common multiple of the minimal polynomials of α^i :

$$g_{bch}(x) = \text{lcm}(M_1(x), M_2(x), \dots, M_{2t}(x)) \quad (12)$$

The inner code receives the Reed–Solomon encoded bitstream, splits it into blocks of length k , and appends $n - k$ parity bits. It corrects up to $t_{bch} = 8$ scattered, independent bit flips per block.

E. HMAC Keyed Placement Offset Engine

To distribute the watermark bits pseudorandomly, the system identifies all word boundaries (spaces and punctuation breaks) in canonicalized plaintext. Let $B = \{b_0, b_1, \dots, b_{M-1}\}$ denote the sorted indices of these boundaries. We derive a pseudorandom index sequence $I = \{i_0, i_1, \dots, i_{L-1}\}$ using an HMAC-SHA256 generator initialized with a server secret K_{key} :

$$\text{seed}_0 = \text{HMAC-SHA256}(K_{key}, H \parallel \text{nonce}) \quad (13)$$

For each bit $j \in \{0, 1, \dots, L - 1\}$, the generator computes:

$$\text{seed}_j = \text{HMAC-SHA256}(K_{key}, \text{seed}_{j-1}) \quad (14)$$

$$r_j = \text{int}(\text{seed}_j[0..7]) / 2^{32} \quad (15)$$

$$\text{idx}_j = \lfloor r_j \times |B| \rfloor \quad (16)$$

The placement positions $P_{place} = \{B[\text{idx}_0], B[\text{idx}_1], \dots, B[\text{idx}_{L-1}]\}$ are sorted in descending order:

$$P'_{place} = \text{SortDescending}(P_{place}) \quad (17)$$

Embedding in descending order prevents insertion offsets from shifting the indices of subsequent boundary positions.

F. Character-Level LCS Alignment

Let $T_{orig} = [a_1, a_2, \dots, a_m]$ be the original watermarked document retrieved from history, and let $T_{recv} = [b_1, b_2, \dots, b_n]$ represent the edited target text. To align the documents, we compute the Longest Common Subsequence (LCS) matrix $DP \in \mathbb{N}^{(m+1) \times (n+1)}$:

$$DP[i, j] = \begin{cases} DP[i-1, j-1] + 1 & \text{if } a_{i-1} = b_{j-1} \\ \max(DP[i-1, j], DP[i, j-1]) & \text{otherwise} \end{cases} \quad (18)$$

We backtrack from $DP[m, n]$ to identify the matching indices:

$$\mathcal{M} = \{(i, j) \mid a_i = b_j \text{ in the optimal alignment}\} \quad (19)$$

Let W_{orig} be the set of indices in T_{orig} that contain zero-width characters. For each $w \in W_{orig}$:

- If $\exists (w, j) \in \mathcal{M}$, the bit associated with a_w is retrieved from $T_{recv}[j]$ and marked as a **successful match**.
- If no such tuple exists, the bit is marked as an **erasure** (lossy bit), and its position is forwarded to the ECC decoder for correction.

G. Jaccard Similarity Lookup

Before alignment, the original document must be fetched from the history database. Given target text words W_{tgt} and historical document words W_{hist} , the Jaccard similarity is:

$$J(W_{tgt}, W_{hist}) = \frac{|W_{tgt} \cap W_{hist}|}{|W_{tgt} \cup W_{hist}|} \quad (20)$$

The database returns the history entry with the highest similarity score, provided $J \geq 0.3$.

H. Graded Tamper Telemetry Scoring

Rather than returning a binary result, ZeroTrace outputs a quantitative representation of text integrity. The continuous Integrity Score $I_{score} \in [0, 1]$ is defined as:

$$I_{score} = 1.0 - \left(w_1 \frac{C_{bch}}{L_{bits}} + w_2 \frac{C_{rs}}{L_{bytes}} + w_3 \frac{U_{blocks}}{B_{total}} \right) \quad (21)$$

where:

- C_{bch} is the count of bit errors corrected by the BCH inner stage.
- C_{rs} is the count of byte symbol errors corrected by the Reed–Solomon outer stage.
- U_{blocks} is the number of uncorrectable blocks.
- $L_{bits}, L_{bytes}, B_{total}$ represent total bits, bytes, and ECC blocks respectively.
- w_1, w_2, w_3 are weights satisfying $0 < w_1 < w_2 \ll w_3$, indicating that uncorrectable blocks decrease text integrity more than minor bit corrections.

IV. IMPLEMENTATION DETAILS

ZeroTrace was developed in TypeScript and runs on Node.js. It features a separate backend API server and an interactive web interface.

A. REST API Schema

- `POST /api/encode`: Receives a text prompt and model identifier. The server runs an autonomous search agent (utilizing the Vercel AI SDK and Mistral APIs) to retrieve facts via the Tavily Search API. The generated text is passed to the embedding module, which returns the watermarked text and saves the record in `watermark_history.json`.
- `POST /api/decode`: Receives the text to analyze. The system canonicalizes it, checks history using word-level Jaccard similarity, and runs the LCS alignment algorithm. It extracts the zero-width bits, passes them to the concatenated BCH/RS decoders, and verifies the RSA signature. The response contains the recovered provenance metadata and the graded tamper telemetry.

B. ECC Implementation

The ECC components are built from scratch to prevent external library dependency overhead:

- `src/ecc/gf256.ts` computes Galois Field multiplication/division using logarithmic tables.

- `src/ecc/reedSolomon.ts` encodes data block-by-block and decodes them by computing syndrome polynomials and solving the error-locator polynomial via Berlekamp–Massey.
- `src/ecc/bch.ts` handles bit-level syndrome computation and error inversion.

V. EXPERIMENTAL RESULTS AND DISCUSSION

To evaluate the robustness of ZeroTrace, watermarked documents (average length: 1,500 characters, containing a 1,200-bit watermark signature) were subjected to different editing attacks. We measured the Bit Error Rate (BER) before decoding, the payload recovery status, signature validation, and the final Integrity Score. The comparison with other watermarking strategies is detailed in Table I.

TABLE I
PERFORMANCE AND FEATURE COMPARISON OF WATERMARKING TECHNIQUES

Feature/Metric	Statistical	Lexical	Unicode Stego	ZeroTrace
Imperceptibility	Poor–Fair	Poor	Excellent	Excellent
Non-Repudiation	No	No	No	Yes (RSA-2048)
Hash-Binding	No	No	No	Yes (SHA-256)
Alignment Support	No	No	No	Yes (LCS)
Corrects Typos	No	No	No	Yes (BCH)
Corrects Deletions	No	No	No	Yes (RS)
Tamper Telemetry	No	No	No	Yes (Graded)

We simulated three attack models to test ZeroTrace’s recovery limits:

- 1) **Scattered Noise Attack (Typos)**: Simulated by randomly flipping characters in the text. As shown in Table II, the inner BCH code corrects up to 8 bit flips per block. The integrity score decreases gracefully as the number of corrections increases.
- 2) **Burst Deletion Attack (Sentence/Paragraph Removal)**: Deleting sentences removes a contiguous sequence of zero-width characters. The LCS alignment detects the missing index range and marks them as erasures. The Reed–Solomon outer code successfully reconstructs the payload for deletions affecting up to 8 bytes of parity symbols per block.
- 3) **Watermark Transfer Attack**: We extracted the zero-width characters from a watermarked AI document and embedded them into a different text. Upon verification, the digital signature was valid, but the decrypted document hash did not match the SHA-256 hash of the modified document. The verification engine correctly raised a warning flag and marked the verification as failed.

VI. ADVANTAGES AND LIMITATIONS

ZeroTrace provides key advantages:

- **High Capacity & Secrecy**: Standard steganography hides data without visible artifacts.

TABLE II
ROBUSTNESS EVALUATION UNDER SIMULATED TEXT ALTERATION
ATTACKS

Alteration Level	Pre-ECC BER	ECC Status	Signature	Integrity Score
0% (Clean)	0.00	Successful	Valid	1.00 (100%)
2% (Scattered)	0.02	Successful	Valid	0.94 (94%)
5% (Scattered)	0.05	Successful	Valid	0.81 (81%)
10% (Burst Delete)	0.09	Successful	Valid	0.74 (74%)
15% (Burst Delete)	0.14	Successful	Valid	0.58 (58%)
25% (Heavy Edit)	0.28	Failed	Invalid	0.00 (0%)

- **Cryptographic Security:** The RSA-SHA256 signature guarantees non-repudiation and prevents provenance forgery.
- **Algorithmic Resilience:** The combination of LCS alignment and concatenated ECC provides robust recovery against realistic editing attacks.

The main limitation is that extreme rewriting or paraphrasing (where more than 30% of the original text is changed or replaced) exceeds the correction bounds of the $BCH(255, 191, 8)$ and Reed–Solomon codecs. In these cases, the watermark cannot be recovered.

VII. CONCLUSION AND FUTURE WORK

This paper presented ZeroTrace, an invisible, cryptographically verifiable, and error-resilient text watermarking framework. By integrating RSA-2048 signing and SHA-256 document binding with a concatenated Reed–Solomon and BCH coding scheme, ZeroTrace ensures content provenance and integrity. The character-level LCS alignment dynamically compensates for frame-shift synchronization errors caused by text deletion, while the error-correcting codes recover the original signature payload.

Future work will focus on:

- Incorporating word embeddings and synonym mapping to handle semantic paraphrasing attacks.
- Designing adaptive ECC scaling that automatically adjusts parity overhead based on document length.
- Deploying decentralized public key registries to facilitate third-party watermark verification.

REFERENCES

- [1] J. Kirchenbauer, J. Geiping, Y. Wen, J. Katz, I. Miers, and T. Goldstein, “A Watermark for Large Language Models,” *International Conference on Machine Learning (ICML)*, 2023.
- [2] I. J. Cox, M. L. Miller, J. A. Bloom, and C. Honsinger, *Digital Watermarking*, San Francisco, CA: Morgan Kaufmann, 2002.
- [3] I. S. Reed and G. Solomon, “Polynomial Codes Over Certain Finite Fields,” *Journal of the Society for Industrial and Applied Mathematics (SIAM)*, vol. 8, no. 2, pp. 300–304, 1960.
- [4] R. C. Bose and D. K. Ray-Chaudhuri, “On a Class of Error Correcting Binary Group Codes,” *Information and Control*, vol. 3, no. 1, pp. 68–79, 1960.
- [5] R. L. Rivest, A. Shamir, and L. Adleman, “A Method for Obtaining Digital Signatures and Public-Key Cryptosystems,” *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, 1978.